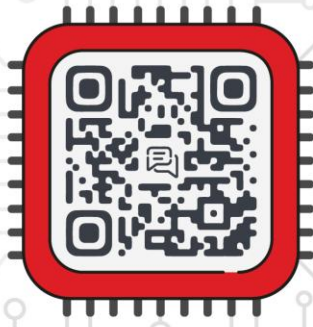




نظري

17

5

2550 sp

كلية الهندسة المعلوماتية

السنة الرابعة

Layers of Architecture & Design Patterns

د. عمار جوخدار



برمجة تفرعية

محتوى مجاني غير مخصص للبيع التجاري
RB Informatics; 27/4/2026

في المحاضرات السابقة، ركّزنا على فكرة أساسية وهي أن تطوير الأنظمة لا يعتمد فقط على المتطلبات الوظيفية (Functional Requirements)، بل بشكل كبير على المتطلبات غير الوظيفية (Non-Functional Requirements) مثل:



- التعقيد العالي
- وصعوبة ضمان الجودة

- الأداء (Performance)
 - الأمان (Security)
 - المتاحية (Availability)
 - التوسع (Scalability)
 - التنافسية (Concurrent Access)
 - وإدارة الموارد (Resource Management)
- كما رأينا أن الأنظمة الحديثة وخاصة التطبيقات الشبكية تتميز بـ:
- تعدد المستخدمين
 - الحاجة إلى التوازي

وفي نهاية المحاضرات السابقة، توصلنا إلى فكرة مهمة:

حل هذه التحديات لا يتم بكتابة كود فقط، بل من خلال تصميم Architecture مناسب للنظام.

لماذا نحتاج إلى البنيان (Architecture)؟

عند البدء بتطوير أي نظام، نواجه التحدي التالي:
هل يمكننا أثناء كتابة كل سطر كود أن نأخذ بعين الاعتبار جميع المتطلبات غير الوظيفية؟

الإجابة عملياً:

هذا صعب جداً، بل يكاد يكون مستحيلاً

وذلك لأن:

- أي سطر في البرنامج يمكن أن يؤثر على الأداء أو الأمان أو التوازي
- التفكير في جميع هذه الجوانب في نفس اللحظة يؤدي إلى تعقيد كبير (يعتبر معضلة كبيرة)

هنا يأتي دور البنيان (Architecture) كحل أساسي لهذه المشكلة:

البنيان هو الطريقة التي نستخدمها لتنظيم النظام بحيث نفصل المتطلبات غير الوظيفية ونديرها بشكل منهجي حيث يساعد البنيان على:

▪ فصل الاهتمامات (Separation of Concerns)

بحيث لا نخلط كل شيء في نفس المكان

▪ استخدام نماذج تصميم (Design Patterns)

لمعالجة كل نوع من المشاكل بشكل مستقل

▪ النتيجة عندما يكون لدينا Architecture جيد:

يمكننا تطوير نظام:

▪ ذو نوعية عالية (Quality).

▪ أكثر استقراراً.

▪ وأسهل في الصيانة والتطوير.

و تصبح جودة النظام أقل اعتماداً على مهارة المبرمج الفردية، وأكثر اعتماداً على التصميم الصحيح للنظام.

ملاحظة هامة:

من النقاط الأساسية التي يجب الانتباه لها في تصميم الأنظمة:

أن أي نظام لا يتم اختباره مسبقاً باستخدام **Benchmarking** على عدد الطلبات (Requests) المتوقع، يصبح من الصعب وأحياناً من المستحيل تصحيح مشاكله لاحقاً على مستوى **Enterprise Architecture** (البنيان المؤسسي). فإذا كان التصميم الأساسي خاطئاً، فإن هذا الخطأ لا يمكن إصلاحه بسهولة لاحقاً، لأنه يكون متجذراً في بنية النظام.

كما يجب التأكيد على أن:

ال Server-Side هو الأساس الحقيقي للنظام

ولا ينبغي الانخداع بالواجهة (UI)، لأن عمل المهندس الحقيقي يكون في "الكواليس"، حيث تتم معالجة التعقيد الحقيقي للنظام، من أداء وتزامن وإدارة موارد، وهي الجوانب الأصعب وغير المرئية للمستخدم. وفي ظل التطور الحالي وظهور أدوات **Low-Code**، أصبح من السهل بناء تطبيقات بسرعة، لكن التحدي الحقيقي هو:

كيف نبني نظاماً يحقق المتطلبات غير الوظيفية دون أن يصبح معقداً أو صعب الصيانة؟

وهنا تظهر مشكلة معروفة:

نعلم أن النوعية (Quality) غالباً ما تتعارض مع:

▪ الزمن (Time)

▪ الكلفة (Cost)

أن المتطلبات غير الوظيفية نفسها قد تتعارض فيما بينها

فمثلاً:

■ كيف نصمم نظاماً:

■ سهل الصيانة

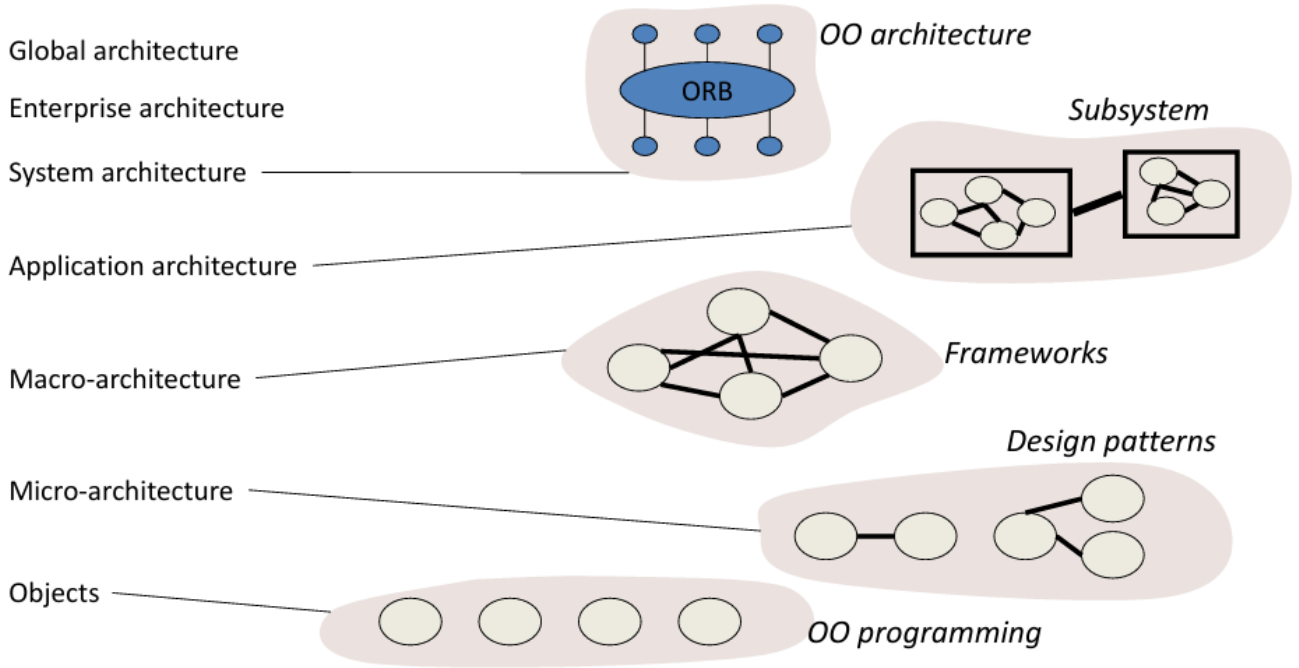
■ وبنفس الوقت بسيط وصغير (عدد أسطر قليل)؟

هذه التناقضات تجعل تصميم البنيان عملية دقيقة تتطلب توازناً مستمراً بين عدة عوامل.

المستويات السبعة للبنيان

البنيان ليس مستوى واحداً، بل هو مجموعة من المستويات (Layers) التي ننتقل بينها من العام إلى الخاص...
تم تقسيم البنيان إلى عدة مستويات كما ورد في مرجع [Mowbray and Malveau](#)، وهي:

1. Global Architecture.
2. Enterprise Architecture.
3. System Architecture.
4. Application Architecture.
5. Macro-Architecture.
6. Micro-Architecture.
7. OO Architecture (Object-Oriented Level).



* Mowbray and Malveau

■ و سنتحدث عن كل مستوى فيما يلي...

OOP

- لماذا نعتبر OOP هو المستوى الأدنى في البنيان؟

قد يبدو غريباً أن نبدأ من **Object-Oriented Programming (OOP)** كأدنى مستوى، مع أن البنيان الحقيقي للحاسوب يبدأ فعلياً من:



- Motherboard

- Bus

- CPU

- وحتى داخل المعالج نفسه يوجد Architecture

التفسير:

في الحقيقة، هذه المستويات **ما زالت موجودة ومهّمة**، لكن:

مستوى التجريد (Abstraction) في الأنظمة الحديثة أصبح عاليًا جدًا

أي أن المبرمج اليوم:

- لا يحتاج أن يفكر مباشرة في:

- أين تُخزن البيانات في الذاكرة

- كيف تنتقل عبر الـ Bus

- أو كيف يعمل المعالج داخليًا

مقارنة مع الماضي:

في السابق:

- كان المبرمج يتعامل مباشرة مع الذاكرة:

- RAM محدودة مثل (64KB)

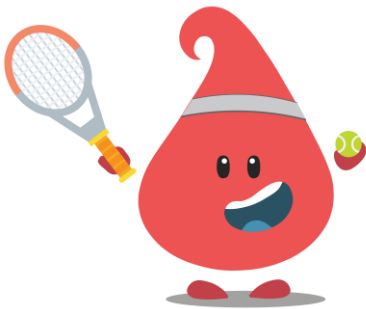
- والباقي على القرص (Hard Disk)

- وكان عليه التفكير بتفاصيل دقيقة جدًا في التخزين

- أما اليوم:

- هذه التفاصيل مخفية خلف طبقات من التجريد.

- والنظام يديرها تلقائيًا.



لكن ... هل نتجاهل البنية التحتية تمامًا؟

الإجابة لا...

رغم أن التجريد أخفى هذه التفاصيل، إلا أن تأثيرها ما زال موجودًا.

مثال مهم:

لا يمكن التعامل مع الـ RAM والـ Hard Disk بنفس الطريقة

لأن:

- RAM سريعة جدًا.
- Hard Disk بطيء نسبيًا.
- لذلك:

لا يجب السماح للنظام أن يتعامل مع الذاكرة بشكل عشوائي.

مثل الاعتماد الزائد على:

- Virtual Memory
- أو ترك الـ RAM "تتمدد" إلى القرص.

فهذا يؤدي إلى:

- انخفاض كبير في الأداء.
- ويُعتبر تصميمًا سيئًا (كارثيًا).

ما الحل إذن؟

هنا يأتي دور التصميم الصحيح:

- إذا احتجنا تخزين دائم => نستخدم Database (على القرص)
- إذا احتجنا سرعة => نستخدم RAM (Caching / In-Memory)

أي أننا:

نتخذ قرارات معمارية واعية بدل ترك النظام يتصرف بشكل عشوائي.

في سياق هذه المادة، نحن نفترض أن:

المبرمج يعمل ضمن مستوى تجريد معين، ولا يتعامل مباشرة مع العتاد (Hardware)

لذلك:

نعتبر OOP هو أدنى مستوى نتعامل معه.

لأنه:

- أقرب مستوى فعلي يكتب فيه المبرمج الكود.
- وهو الأساس الذي تُبنى عليه باقي المستويات.

إضاءة أريسيزية: المبرمج الجيد يكتب كود جيد، لكن المهندس الجيد يفكر في النظام على جميع المستويات

أنماط التصميم Design Patterns

بعد أن تعرّفنا على أدنى مستويات البنيان، نصل إلى مستوى مهم جداً وهو: Design Pattern وهي حلول جاهزة ومجربة لمشاكل متكررة في تصميم البرمجيات.

■ مثال : MVC (Model View Controller)

يُعتبر MVC من:

■ أشهر Design Patterns.

■ وفي نفس الوقت من أصعبها تطبيقاً بشكل صحيح.

الفكرة الأساسية لـ MVC:

يقوم هذا النمط على فصل مكونات التطبيق إلى ثلاثة أجزاء:

■ Model: **منطق العمل (Business Logic) + البيانات.**

■ View: الواجهة (UI).

■ Controller: وسيط يربط بينهما.

■ المشكلة سابقاً:

في الماضي، كان من الصعب على المطورين

تطبيق هذا الفصل بشكل صحيح، لأن:

■ كانوا يضعون منطق العمل داخل الواجهة (View).

■ مثل:

■ التحقق من الصلاحيات (Security Checks)

■ معالجة البيانات

وهذا خطأ كبير

لأن الواجهة لا يجب أن تحتوي على أي منطق أمني أو منطقي

■ لماذا هذا خطأ؟

عندما نخلط بين الواجهة و منطق العمل نحصل على: كود معقد / صعب الصيانة / غير قابل للتوسعة

■ ماذا تغير اليوم؟

مع تطوّر الأدوات الحديثة:

■ ظهرت أدوات توليد الواجهات (UI Generators / Frameworks)

■ وأصبح تطبيق MVC :

■ أسهل

■ وأكثر التزاماً بالمفهوم الصحيح

أي أن: الفصل بين Model و View أصبح أوضح وأدق.

فائدة أربيسيزية:

قد يحدث التباس عند قراءة العبارة:

Model = Business Logic + Data

لأن كثيراً من المبرمجين يتعلمون عملياً أن: "الـ Logic موجود داخل الـ Controller" **لكن في الحقيقة، يجب التمييز بين نوعين من الـ logic**

1. **منطق العمل الحقيقي (Business Logic) وهو:**

القواعد الأساسية التي تعبر عن طريقة عمل النظام نفسه

مثل: حساب الرصيد البنكي، التحقق من صلاحية عملية شراء،

حساب الضريبة، شروط نجاح العملية أو فشلها

هذا النوع يجب أن يكون داخل: Model / Services

لأنه يمثل جوهر النظام **ويجب أن يبقى مستقلاً** عن الواجهة وطريقة الطلب

2. **منطق التحكم (Control Logic):**

وهو المنطق المرتبط باستقبال الطلبات، توجيه البيانات،

استدعاء الخدمات المناسبة، إعادة النتائج للواجهة، وهذا هو

دور Controller أي أن الـ Controller لا يحتوي منطق

العمل الحقيقي **بل ينسّق تدفق الطلب فقط.**



■ أمثلة أخرى على Design Patterns في السلايد:

- Factory: لإنشاء الكائنات مثل (DB Connection)
- DAO: للوصول إلى البيانات
- ORM: ربط الكائنات مع قواعد البيانات
- Observer: لمراقبة التغييرات (Listeners)
- Facade: تبسيط التعامل مع أنظمة معقدة
- AOP: فصل المتطلبات غير الوظيفية عن الوظيفية
- SOA: تصميم النظام على شكل خدمات
- Strategy: يسمح باستخدام وتبديل أكثر من طريقة لتنفيذ نفس المهمة بدون تعديل الكود.
- MDA: بناء نماذج قبل كتابة الكود الفعلي، لضمان أن التصميم يطابق احتياجات الشركة.
- Lazy List: عدم تحميل البيانات مرة واحدة لتوفير الموارد
- Message: أسلوب تواصل بين أجزاء النظام.
- Frames: هي الهياكل التي تنظم عرض البيانات أو العمليات داخل النظام.

Frameworks vs Design Patterns

بعد أن تعرّفنا على Design Patterns، ننتقل إلى الطبقة الأعلى وهي: Frameworks

■ العلاقة بين Design Patterns و Frameworks

■ Design Patterns

- حلول لمشاكل محددة.
- مستوى منخفض نسبياً (Low-Level).

■ Frameworks

- مستوى أعلى.
- تقوم على تجميع عدة Design Patterns معاً.
- وتقدم هيكلية جاهزة لبناء التطبيقات.

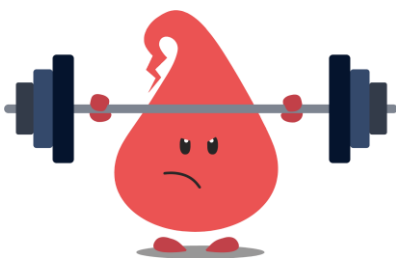
■ بمعنى:

Framework = مجموعة Design Patterns تعمل معاً بشكل منسق.

نقطة مهمة جداً: لا يكفي استخدام Design Patterns بشكل عشوائي.

بل يجب أن يكون هناك توافق (Compatibility) بينها وإلا:

- لا يمكن دمجها بشكل صحيح وقد يؤدي ذلك إلى:
- تعقيد / أخطاء / وفشل في النظام



■ مشكلة واقعية:

في الواقع، كثير من المطورين:
لا يستخدمون Design Patterns بشكل صحيح وهذا ما أدى إلى مشاكل كبيرة، خاصة مع ظهور Frameworks قوية.
مثال Spring:

عند ظهور Spring Framework :

■ حاولت الشركات استخدامه بكثافة لكنه:

■ معقد.

■ ويعتمد على فهم عميق لـ Design Patterns.

النتيجة: عدد كبير من الشركات **فشل** في استخدامه بشكل صحيح.

■ كيف تم حل المشكلة؟

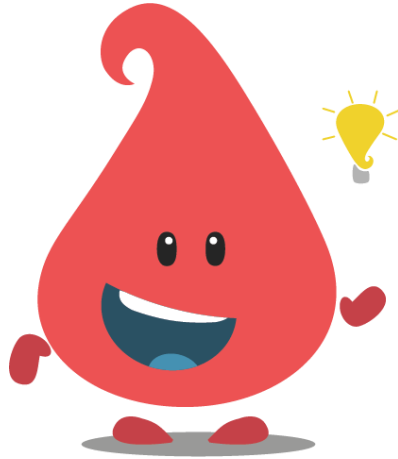
تم تقديم أدوات أعلى مستوى مثل:

■ Spring Boot

■ Builders / Starters

الهدف منها:

تبسيط استخدام الـ Framework وإخفاء التعقيد عن المطور.



■ مفهوم مهم: نوعان من الـ Frameworks

في منصات مثل Java Enterprise :

تم التمييز بين:

1. Technical Framework

■ يقدم الأدوات والتقنيات مثل:

■ إدارة التزامن

■ التعامل مع قواعد البيانات

■ إدارة الطلبات

2. Software Engineering Framework

■ يقدم طريقة الاستخدام أي:

كيف نستخدم الـ Technical Framework بشكل صحيح، أي هو عبارة عن قواعد و إرشادات و Best Practices.

Don't shrink your goals increase your effort...

▪ مشكلة الاستخدام المفرط في Spring مثلاً:

▪ كان هناك اعتماد كبير على: Enterprise Components

حتى في حالات لا تحتاج هذا التعقيد فكان الحل: العودة إلى البساطة و عندها ظهر مفهوم:

POJO (Plain Old Java Objects) الفكرة:

- استخدام كائنات عادية.
- بدون تعقيد.
- بدون Framework ثقيل.



Application Architecture

بعد أن تعرّفنا على Frameworks، ننتقل الآن إلى مستوى أعلى في البنية: Application Architecture في هذا المستوى نقوم بـ:

تقسيم النظام إلى مجموعة من Subsystems

▪ لماذا نستخدم Subsystems؟

إذا كان لدينا برنامج عبارة عن:

كتلة واحدة (Monolithic) من 100,000 سطر

ثم ظهر خطأ سيكون:

- من الصعب جداً معرفة مصدر الخطأ
- التعقيد عالي جداً

▪ ماذا يحدث عند استخدام Subsystems؟

عند تقسيم النظام إلى Subsystems كل Subsystem :

- له حدود واضحة.
- له مدخلات ومخرجات معروفة.

وهذا هو الفرق الأساسي بين:

- Subsystem
- Module عادي (مجرّأ) قد لا تكون حدوده واضحة.

▪ تأثير التقسيم على التعقيد:

▪ بدون Subsystems :

التعقيد = ضرب (Multiplication) ، لأن كل جزء قد يتفاعل مع كل جزء آخر.



▪ مع Subsystems :

التعقيد = جمع (Addition)، لأن كل Subsystem مستقل نسبيًا.

▪ تأثيره على الاختبار (Testing):

▪ مع Subsystems :

▪ يمكن إجراء اختبارات مستقلة 100% لكل Subsystem (Unit / Subsystem Testing)

▪ بدون Subsystems :

نضطر إلى:

▪ Integration Testing فقط (اختبار النظام ككل) وهذا أصعب وأكثر تعقيدًا.

▪ الخلاصة

▪ Subsystems هي إحدى أهم الطرق لتقليل التعقيد في التطبيقات، خاصة تطبيقات الويب.

System Architecture

بعد تقسيم النظام إلى Subsystems، ننتقل للسؤال الأهم:

▪ كيف ستتخاطب هذه Subsystems مع بعضها؟

هل:

▪ كل Subsystem يتواصل مع الآخر مباشرة؟

▪ أم يوجد وسيط؟

▪ ما هي طريقة التواصل؟

هذه القرارات تتخذ في:

مرحلة البنيان (Architecture Phase) وليس بعد بدء التنفيذ.

▪ ما هو ال Middleware:

هو وسيط التواصل بين Subsystems وهو المسؤول عن:

▪ نقل البيانات.

▪ تنظيم الاتصال.

▪ إدارة التفاعل بين الأجزاء المختلفة.

و منه :

System Architecture = تصميم طريقة التواصل بين Subsystems باستخدام Middleware.

التأكيد على فكرة مهمة جدًا:

قرار التكامل (Integration) يجب أن يُتخذ من البداية

لا يجب أن: "يُجبرك عمل الآخرين على تغيير تصميمك لاحقًا"

خطأ شائع

أن نبدأ ببناء SubSystems

ثم لاحقاً نفكر: "كيف سنربطها مع بعضها؟"

هذا يؤدي إلى: إعادة تصميم - مشاكل

تكامل - وتأخير كبير

- Application Architecture <= تقسيم النظام إلى Subsystems
- System Architecture <= تحديد كيف تتواصل هذه Subsystems

Enterprise Architecture

بعد أن حددنا:

- Application Architecture
- System Architecture

ننتقل إلى مستوى أعلى يظهر عندما:

- يكون النظام مؤسساتيًا (Enterprise System)
- متى نحتاج هذا المستوى؟

عندما يصبح النظام:

- كبير الحجم - يخدم عددًا كبيراً من المستخدمين
- ويتعامل مع:
- أداء عالي (Performance)
- إدارة موارد (Resource Management)
- توازي (Concurrency)
- متاحة (Availability)

أي أننا ندخل بقوة في المتطلبات غير الوظيفية.

- دور Enterprise Architecture:

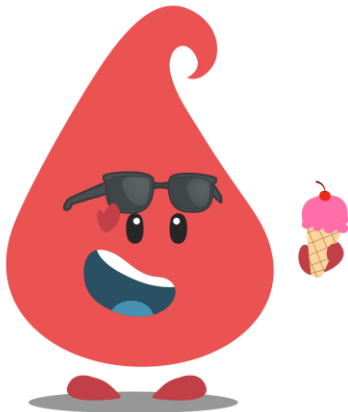
الهدف الأساسي لهذا المستوى هو تحقيق المتطلبات غير الوظيفية على مستوى النظام ككل مع الأخذ بعين الاعتبار أنه يبنى فوق الطبقات السابقة.

ماذا يضيف هذا المستوى؟

- قرارات تتعلق بـ:

- توزيع الحمل (Load Distribution)
- التوسع (Scalability)
- إدارة الموارد
- الاعتمادية (Reliability)

أي أنه يركّز على: جعل النظام يعمل بكفاءة في بيئة حقيقية.



Global Architecture

وهو أعلى مستوى في البنيان

▪ متى نحتاجه؟

عندما يكون لدينا:

أكثر من نظام مؤسساتي (Enterprise Systems) تتواصل مع بعضها.

أمثلة:

▪ نظام بورصة يتواصل مع نظام مصرف.

▪ نظام دفع إلكتروني يتكامل مع: بنوك / شركات اتصالات / أنظمة حكومية.

دور Global Architecture:

▪ تنظيم التواصل بين عدة Enterprise Architectures.

الخلاصة النهائية للمستويات

نكون قد وصلنا إلى التسلسل الكامل:

▪ OOP → كتابة الكود

▪ Design Patterns → تنظيم الكود

▪ Frameworks → تجميع الأنماط

▪ Application Architecture → تقسيم النظام

▪ System Architecture → ربط الأجزاء

▪ Enterprise Architecture → تحقيق المتطلبات غير الوظيفية

▪ Global Architecture → ربط الأنظمة الكبرى

Design Pattern: Meta Data

أحد المفاهيم التي تطورت كثيراً و أصبحت مدعومة في معظم لغات OOP الحديثة هي :

Meta Data (البيانات الوصفية عن البيانات).

🔗 ما المقصود بالـ Meta Data؟

هي ببساطة: معلومات عن الكود نفسه.

أي أصبح بإمكاننا أثناء التنفيذ أن نسأل:

▪ ما نوع الكلاس X؟

▪ ما هي الحقول (Fields) الموجودة فيه؟

▪ ما نوع كل حقل؟

▪ ما هي التوابع (Methods)؟

▪ ما هي بارامتراتهما وأنماطهما؟

هذا ما يُعرف بـ: Introspection.

■ ماذا أضافت Meta Data؟

في السابق:

كان الكود مرتبطاً بالنمط (Type) بشكل صارم مثل:

`sort<int>` مختلف عن `sort<string>`.

أما الآن:

يمكننا كتابة كود مجرد (Abstract) يعمل على أي نوع

مثل التعامل مع الكائنات بدون معرفة نوعها مسبقاً

■ التطور عبر الزمن:

■ في الماضي:

لغات مثل ++C و Java كانت محدودة في هذا المجال.

بينما:

■ لغات مثل Prolog و Lisp كانت تدعم هذه المفاهيم بشكل أقوى.

■ لذلك سُمّيت: "لغات الذكاء الصناعي".

■ خلاصة

Meta Data نقلتنا من:

■ كود مرتبط بالأنماط (Type-dependent)

■ إلى:

■ كود مجرد (Abstract).

■ قادر على التعامل مع أي نوع بيانات.

Design pattern: Observer / Listeners

أحد الأنماط المهمة المرتبطة بإدارة السلوك في الأنظمة هو: Observer Pattern

ويُستخدم كثيراً فيما يُعرف بـ:

■ Listeners.

■ أو في تطبيق Business Rules.

الفكرة الأساسية:

بدلاً من أن نقوم بتعديل الكود مباشرة، ونضع شروطاً مثل إذا حدث X نفذ Y

يمكننا استخدام Listeners بحيث:

نستمع (Listen) لحدث معين، وعند حدوثه يتم تنفيذ كود محدد.

لماذا هذا مهم؟

في الطريقة التقليدية بدون (Observer) :

- إذا أردنا إضافة سلوك جديد:
 - يجب أن نبحث عن كل مكان يتم فيه استدعاء التابع.
 - ونضيف شرطاً في كل مكان.
- هذا يؤدي إلى: تكرار الكود - صعوبة الصيانة - احتمال حدوث أخطاء

باستخدام Observer / Listeners:

- نقوم بتعريف Listener واحد يرتبط بحدث معين.
- وعند حدوث الحدث:
- يتم تنفيذ الكود تلقائياً بدون تعديل الكود الأصلي.



مفهوم ال Aspect

ال Aspect هو:

وحدة تحتوي على سلوك معين نريد إضافته للنظام بدون تعديل الكود الأساسي

■ مثل تنفيذ كود عند:

- بداية تابع.
- نهاية تابع.
- أو عند حدوث حدث معين.

هذه الفكرة تعتبر نقلة **مهمّة** لأنها تغيّر طريقة التفكير من "**أين أضف الكود؟**" إلى "**متى يتم تنفيذ هذا الكود؟**".

مثال من الواقع:

بدلاً من كتابة في كل تابع تحقق من الصلاحيات يمكننا:

- إنشاء Aspect للأمان (Security).
- يتم تطبيقه تلقائياً على التوابع المطلوبة.

ملاحظة:

هذه المفاهيم مستخدمة **بشكل واضح** في: Java خصوصاً مع (Spring AOP).
وتظهر أيضاً في JavaScript باستخدام مفاهيم مثل: Proxy.

خلاصة المحاضرة

■ Design Pattern هو حل لمشكلة شائعة ومتكررة في البرمجة

كما أنه:

- مستوى أدنى من Frameworks.
- حيث أن Framework يستخدم مجموعة من Design Patterns معاً بشكل منسق.

■ تعريف التحليل (Analysis)

هو وصف النظام من وجهة نظر المستخدم النهائي

- أي أنه يركز على كيف يتفاعل المستخدم مع النظام
- ما هي:
- المدخلات (Inputs)
- المخرجات (Outputs)

كل ما يستطيع المستخدم فهمه أو استخدامه يُعتبر جزءاً من التحليل

لذلك **أهم** Diagram في مرحلة التحليل هو Use Case Diagram

لأن التحليل يُعنى بـ **الاستخدام** (Usage).

■ نتيجة التحليل:

تبدأ الصورة بالتوضّح حول:

- الكلاسات (Classes).
- الحقول (Fields).
- والعلاقات بينها.

■ تعريف التصميم (Design)

هو إسقاط التحليل على البنيان (Architecture) **أي تحويل** ما فهمناه من المستخدم **إلى بنية** تقنية قابلة للتنفيذ.

■ ما الذي يميّز تصميمًا عن آخر؟

الفرق الأساسي بين التصميمين يكون **في** مدى تحقيقها للمتطلبات غير الوظيفية.

■ الهدف الأساسي من التصميم هو:

تحقيق المتطلبات غير الوظيفية

ملاحظة مهمة:

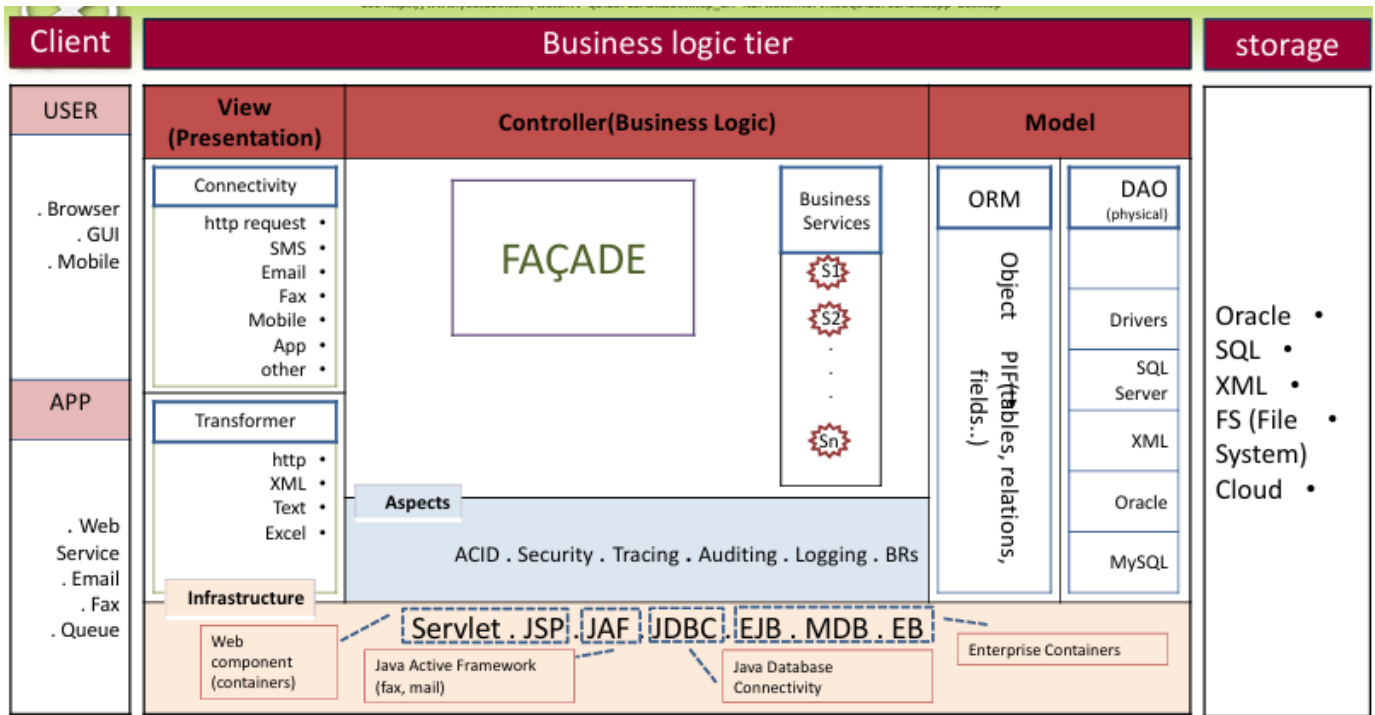
هذه المفاهيم ليست نظرية فقط، بل هي أساس المادة بالكامل (Core Concepts).

تمهيد للمحاضرة القادمة

3-Tiers Design Pattern

فكرة أساسية سنبنى عليها في المحاضرة القادمة، وهي

أول Design Pattern على مستوى البنية المؤسساتي هو 3-Tiers Architecture



ما هو 3-Tiers ؟

يقوم هذا النمط على تقسيم التطبيق إلى 3 طبقات (Tiers) :

1. Client Tier طبقة العميل.

2. Business Logic Tier طبقة منطق العمل.

3. Storage Tier طبقة التخزين.

■ ماذا نقصد بـ Tier ؟

Tier = Subsystem

أي أن كل Tier هو:

- جزء مستقل نسبياً من النظام.
- له دور محدد.
- ويتواصل مع باقي الأجزاء بطريقة منظّمة.

↳ نظرة على طبقة Client:

في طبقة Client يمكن أن نجد:

Client
USER
. Browser . GUI . Mobile
APP
. Web Service . Email . Fax . Queue

User المستخدم:	■
Browser	■
GUI	■
Mobile	■
أو Application :	■
Web Service	■
Email	■
Queue	■

أي أنها تمثل نقطة التفاعل مع النظام.

إذا كنت تظن أن التعليم مكلف، فجرب الجهل

نكمل في المحاضرة القادمة...

